DOCKER

By Chaitanya Raj Nandan

Contents

1 Before Containers 3
1.1 What is A Container? 3
1.2 Containers Vs Virtual Machines 3
1.3 Advantages of Containerization 4
2 Introduction 5
2.1 Why Use Docker 5
2.2 Docker Terminologies / Docker Architecture 5
3 Dockerfiles 7
4 Install Docker in Linux (Ubuntu) 9
5 Uninstall & Remove DOCKER 12
6 Basic Docker Commands 14
6.1 Docker PS 14
6.2 Docker Pull 14
6.3 Docker Images 16
6.4 Docker Run 16
6.5 Docker PS 18
6.6 Docker Stop 19
6.7 Docker RM (Remove Container) 19
6.8 Docker RMI (Remove Image) 19
6.9 Docker Build 20
6.10 Docker Push 20
6.11 Docker Exec 20
7 Building a Python Web App 21
8 ERROR AND ITS SOLUTION 22
8.1 Unauthorized: incorrect username or password 22

1 Before Containers

Let us dial back the clock. Back when computers were a novelty, and companies wanted to run their applications, they would have had to buy a server for doing so. However, this came with a caveat: you could only run one application per server. If you wanted to run multiple applications, you would have to buy as many servers. As costs would skyrocket, this was a problem, for both the company and the environment on a larger scale

IBM fixed this issue by introducing the concept of *virtual machines* into the game. With them, we could run multiple applications on the same server. You may have heard of / done dual booting on your PC; like installing windows on your Mac device or Ubuntu on Windows. These are virtual machines. However, they had a problem as well. They needed their own operating system, which tended to take up a lot of memory and storage on your hard disk. This made them slow and not so efficient.

In came containers.

1.1 What is A Container?

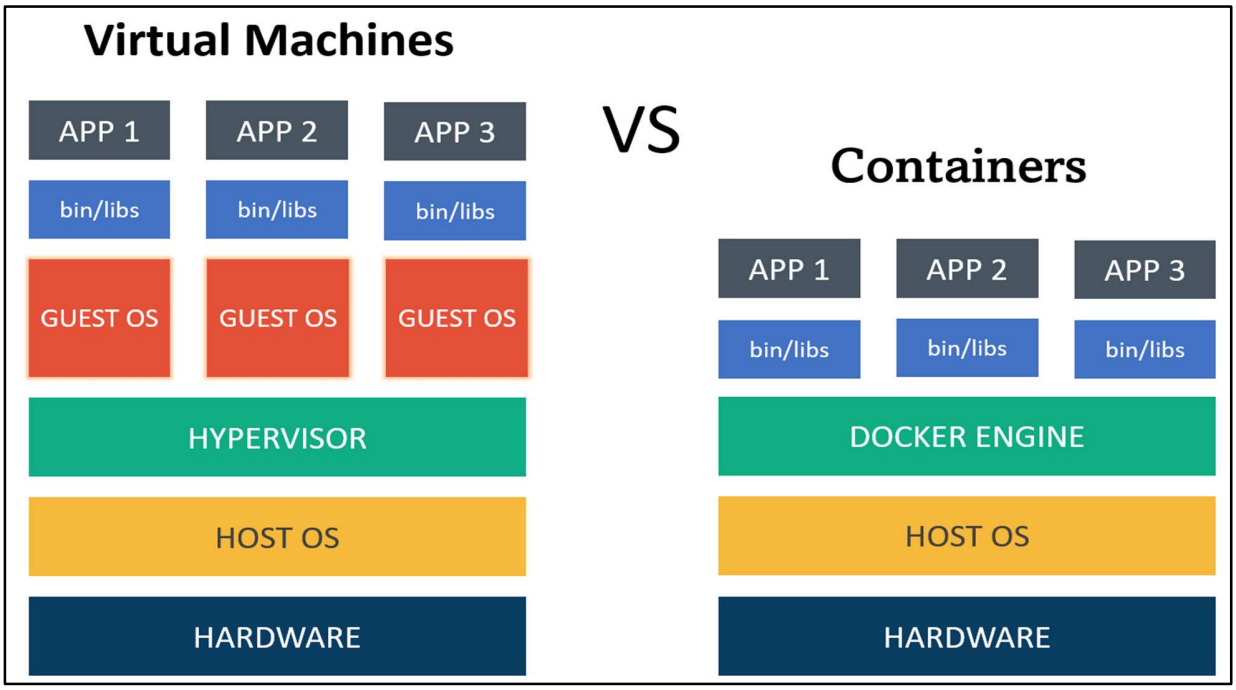
Imagine you are moving to US from India. Would it make sense to send your belongings one by one, or would you rather put them all in one giant box and ship the box directly? Obviously, the latter, right?

That big box is, in simple terms, a container.

Let's say you built a website that works well on your system but your friend runs into some problems with it when they try using it on their computer. To avoid such hassles, you can use a container, which would ship the entire website along with its dependencies, such as the web database, front end, back end, source code, etc. This would ensure that the website runs smoothly.

In computing terms, containerization is an efficient method for running, deploying and scaling applications.

1.2 Containers Vs Virtual Machines



Any application which runs on a virtual machine will require a guest OS to run, which would require a hypervisor. A hypervisor manages the virtual machines and is used to create multiple machines on the host operating system. As you can see from the picture, every OS would require a dedicated amount of space in the hardware, which is *virtually* divided.

In the case of containers, you need have only one operating system, and on top of that, you have a container engine, on which you run applications. They use the idea of isolating your application from the main operating system.

To sum it up succinctly, virtual machines use multiple operating systems to run multiple applications whereas containers use only the host operating system to run applications via the container engine.

In reality, however, containers run on top of virtual machines. This is because virtual machines allow faster access to hardware resources and hypervisors provide better security compared to containers.

Aspect	**Containers**	**Virtual Machines**
OS	Share host's Kernel	Has its own Kernel
Resource Usage	Lightweight, efficient	Heavier, more resource usage
Startup Time	Quick start	Slower start
Isolation	Process-level separation	Full OS isolation
Portability	Highly portable	Compatibility concerns
Resource Overhead	Minimal overhead	Higher overhead
Isolation Level	Lighter isolation	Stronger isolation

1.3 Advantages of Containerization

- Increased Portability
- Easier Scalability
- Easy and Fast Deployments
- Better Productivity
- Improved Security
- Consistent test environment for development and QA.
- Cross-platform packages called images.
- Isolation and encapsulation of application dependencies.
- Ability to scale efficiently, easily, and in real time.
- Enhances efficiency via easy reuse of images.

2 Introduction

Docker is a platform for developing, shipping, and running applications in containers. Containers are lightweight, portable, and self-sufficient units that can run applications and their dependencies in isolated environments. Docker provides a consistent and reproducible environment across different machines, making it easier to manage and deploy applications.

2.1 Why Use Docker

- Helps transport code faster.
- Makes scaling, deploying, and identifying issues in the application easier.
- Saves up space as you do not need to install the whole application locally.

2.2 Docker Terminologies / Docker Architecture

Docker Engine: The core of Docker is the Docker Engine, which is responsible for building, running, and managing containers. It consists of a server (daemon) and a REST API that allows you to interact with the daemon. It is the heart of the Docker architecture, which does the crucial work of building, running, and distributing the containers. It also manages the Docker images and the containers.

- ✓ Server – which runs the daemon
- ✓ Rest API – deals with the interaction of applications with their server
- ✓ Client – which is nothing but the command line interface (CLI)

Docker Images: Images are lightweight, standalone, and executable packages that include everything needed to run a piece of software, including the code, runtime, libraries, and system tools. Images are used to create containers.

Docker Containers: Containers are instances of Docker images that run in isolated environments. They encapsulate an application and its dependencies, ensuring consistency across different environments. Containers are portable and can run on any system that supports Docker.

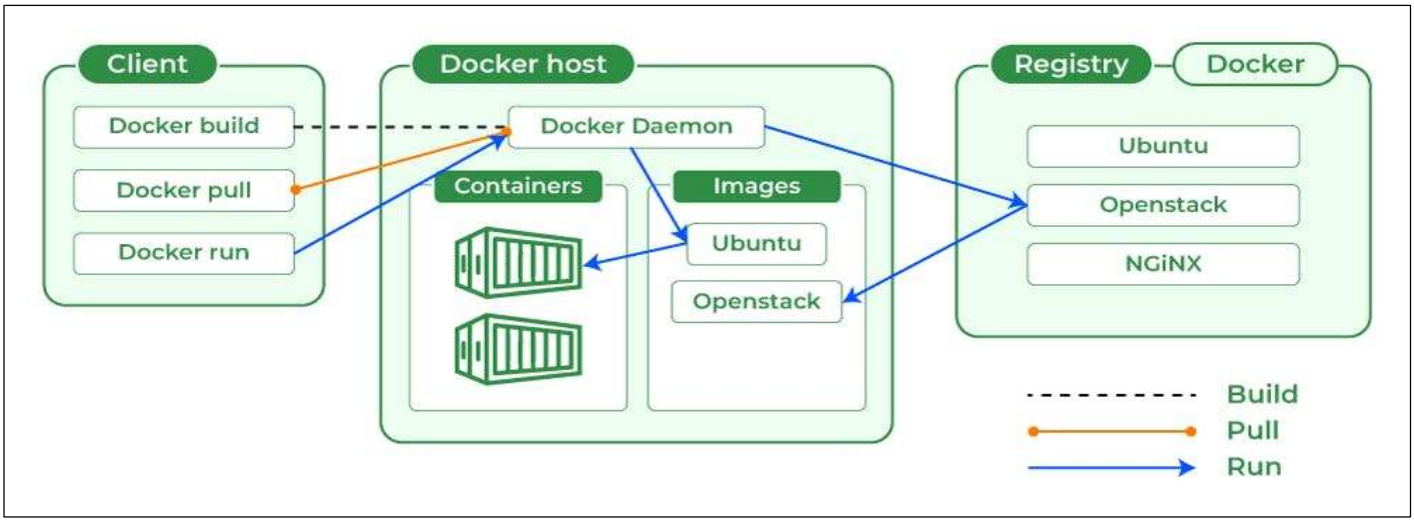
Docker Runtime: It allows us to start and stop containers. There are two types:

- Low-level runtime (runc)
- High-level runtime (containerd)

Docker Client: It is what we use to interact with Docker. You can think of it like the user interface for Docker. Whenever you type in a command, the client sends these to the daemon.

Docker CLI: It allows users to issue commands to the Docker Daemon. Docker uses a client – server architecture.

Docker Registries: Docker Registries store Docker images. The default public registry is Docker Hub, but you can also use private registries or other public registries. Docker Hub is a cloud-based repository where Docker users and partners create, test, store, and distribute container images. It is the default registry used by Docker, making it easy to find and share container images.


Client
Docker build
Docker pull
Docker run
Docker host
Docker Daemon
Containers
Images
Ubuntu
Openstack
Registry
Docker
Ubuntu
Openstack
NGiNX
Build
Pull
Run

3 Dockerfiles

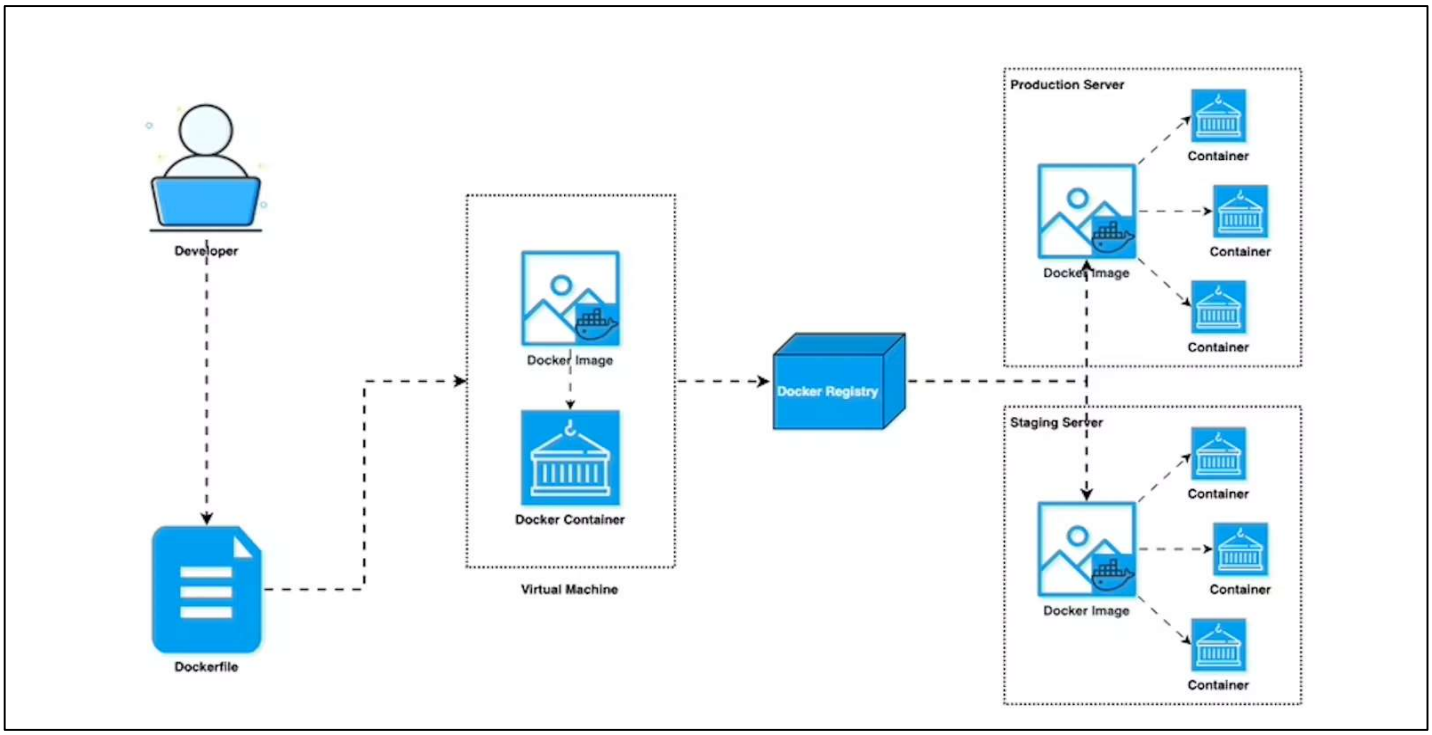
A Dockerfile is a script that contains a set of instructions for building a Docker image. Docker images are lightweight, standalone, and executable packages that include everything needed to run a piece of software, including the code, runtime, libraries, and system tools. Dockerfiles are used to automate the process of creating Docker images.

Here's a basic example of a Dockerfile:

```
# Use an official base image
FROM ubuntu:20.04

# Set the working directory
WORKDIR /app

# Copy application code into the container
COPY . .

# Install dependencies RUN apt-get
update && \     apt-get install -y
python3 && \     apt-get clean

# Expose a port
EXPOSE 8080

# Define the command to run the application
CMD ["python3", "app.py"]
```

Now, let's break down each part of this Dockerfile:

- **FROM:** Specifies the base image. In this case, it's Ubuntu 20.04.
- **WORKDIR:** Sets the working directory inside the container. Subsequent instructions will be executed from this directory.
- **COPY:** Copies files from the host machine to the container. Here, it copies the content of the current directory (the "." represents the current directory) into the "/app" directory inside the container.
- **RUN:** Executes commands during the image build process. In this example, it updates the package list, installs Python3, and cleans up the package cache.
- **EXPOSE:** Informs Docker that the container will listen on the specified network ports at runtime. It doesn't actually publish the ports, but it is a helpful documentation for other developers.
- **CMD:** Provides the default command to run when the container starts. It specifies the command to run our application (in this case, a Python script). Now, let's move on to a multi-stage Dockerfile example. Multi-stage builds allow you to use multiple FROM statements in a single Dockerfile, resulting in a smaller and more efficient final image.


Developer
Dockerfile
Docker Image
Docker Container
Virtual Machine
Docker Registry
Production Server
Container
Container
Container
Docker Image
Staging Server
Container
Container
Container
Docker Image

4 Install Docker in Linux (Ubuntu)

To install Docker using package managers on Ubuntu, you can use the following steps:

1. Check ubuntu version.

lsb_release -a

cat /etc/os-release

```
root@Docker:~# cat /etc/os-release
PRETTY_NAME="Ubuntu 22.04.1 LTS"
NAME="Ubuntu"
VERSION_ID="22.04"
VERSION="22.04.1 LTS (Jammy Jellyfish)"
VERSION_CODENAME=jammy
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=jammy
root@Docker:~# lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 22.04.1 LTS
Release:        22.04
Codename:       jammy
root@Docker:~#
```

2. Update the package cache

   sudo apt update

3. Install the Docker package

   sudo apt install docker.io

```
chetan@Chaitanya-Raj:~/docker$ sudo apt install docker.io
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  bridge-utils containerd dns-root-data dnsmasq-base pigz runc ubuntu-fan
Suggested packages:
  ifupdown aufs-tools btrfs-progs cgroupfs-mount | cgroup-lite debootstrap docker-doc rinse zfs-fuse | zfsutils
The following NEW packages will be installed:
  bridge-utils containerd dns-root-data dnsmasq-base docker.io pigz runc ubuntu-fan
0 upgraded, 8 newly installed, 0 to remove and 0 not upgraded.
Need to get 75.5 MB of archives.
After this operation, 284 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://archive.ubuntu.com/ubuntu jammy/universe amd64 pigz amd64 2.6-1 [63.6 kB]
Get:2 http://archive.ubuntu.com/ubuntu jammy/main amd64 bridge-utils amd64 1.7-1ubuntu3 [34.4 kB]
Get:3 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 runc amd64 1.1.12-0ubuntu2~22.04.1 [8405 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 containerd amd64 1.7.12-0ubuntu2~22.04.1 [37.8 MB]
Get:5 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 dns-root-data all 2023112702~ubuntu0.22.04.1 [5136 B]
Get:6 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 dnsmasq-base amd64 2.90-0ubuntu0.22.04.1 [374 kB]
Get:7 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 docker.io amd64 24.0.7-0ubuntu2~22.04.1 [28.8 MB]
Get:8 http://archive.ubuntu.com/ubuntu jammy/universe amd64 ubuntu-fan all 0.12.16 [35.2 kB]
Fetched 75.5 MB in 16s (4754 kB/s)
Preconfiguring packages ...
Selecting previously unselected package pigz.
(Reading database ... 31638 files and directories currently installed.)
Preparing to unpack .../0-pigz_2.6-1_amd64.deb ...
Unpacking pigz (2.6-1) ...
Selecting previously unselected package bridge-utils.
Preparing to unpack .../1-bridge-utils_1.7-1ubuntu3_amd64.deb ...
Unpacking bridge-utils (1.7-1ubuntu3) ...
Selecting previously unselected package runc.
Preparing to unpack .../2-runc_1.1.12-0ubuntu2~22.04.1_amd64.deb ...
Unpacking runc (1.1.12-0ubuntu2~22.04.1) ...
Selecting previously unselected package containerd.
Preparing to unpack .../3-containerd_1.7.12-0ubuntu2~22.04.1_amd64.deb ...
Unpacking containerd (1.7.12-0ubuntu2~22.04.1) ...
Selecting previously unselected package dns-root-data.
Preparing to unpack .../4-dns-root-data_2023112702~ubuntu0.22.04.1_all.deb ...
Unpacking dns-root-data (2023112702~ubuntu0.22.04.1) ...
Selecting previously unselected package dnsmasq-base.
Preparing to unpack .../5-dnsmasq-base_2.90-0ubuntu0.22.04.1_amd64.deb ...
```

4. Verify the installation

docker version

```
chetan@Chaitanya-Raj:~/docker$ docker version
Client:
 Version:           24.0.7
 API version:       1.43
 Go version:        go1.21.1
 Git commit:        24.0.7-0ubuntu2~22.04.1
 Built:             Wed Mar 13 20:23:54 2024
 OS/Arch:           linux/amd64
 Context:           default
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Get "http://%2Fvar%2F
run%2Fdocker.sock/v1.24/version": dial unix /var/run/docker.sock: connect: permission denied
chetan@Chaitanya-Raj:~/docker$
```

which docker

```
chetan@Chaitanya-Raj:~/docker$ which docker
/usr/bin/docker
chetan@Chaitanya-Raj:~/docker$
```

This command should output the version of Docker that is installed on your system.

5. Add the current user to the docker group\

```
sudo usermod -aG docker ${USER}
```

This will allow you to run Docker commands without having to use sudo.

```
chetan@Chaitanya-Raj:~/docker$ sudo usermod -aG docker chetan
chetan@Chaitanya-Raj:~/docker$ cat /etc/group
```

```
BB:x:1004:
_cvsadmin:x:117:
ssl-cert:x:118:
docker:x:119:chetan
```

6. Log out and log back in

```
sudo reboot
```

This is necessary for the changes to take effect.

Once you have completed these steps, you will be able to use Docker on your Ubuntu system.

5 Uninstall & Remove DOCKER

IMPORTANT!!! If you want to delete everything listed
(**be careful because this is typically not what you actually want**), you can run,

```
sudo find / -name "*docker*" -exec `rm -rf` {} +
```

// check docker is installed or not

```
dpkg -l | grep -i docker
```

// remove volume , network , container and image files

```
sudo docker volume prune -f
sudo docker network prune -f
sudo docker container prune -f
sudo docker image prune -a
```

// remove docker

```
sudo apt purge docker
OR
sudo apt remove docker
```

// remove dependency packages related to docker

```
sudo apt autoremove
sudo apt autoclean
```

```
chetan@Chaitanya-Raj:~/docker$ dpkg -l | grep -i docker
ii  docker.io                        24.0.7-0ubuntu2~22.04.1                amd64        Linux container runtime
chetan@Chaitanya-Raj:~/docker$ sudo docker volume prune -f
Deleted Volumes:
f7be1012d68f2f5de382ef273cb16cb844aca823e72b44d34a8264a964af20c1

Total reclaimed space: 209.1MB
chetan@Chaitanya-Raj:~/docker$ sudo docker network prune -f
chetan@Chaitanya-Raj:~/docker$ sudo docker container prune -f
Total reclaimed space: 0B
chetan@Chaitanya-Raj:~/docker$ sudo docker image prune -a
WARNING! This will remove all images without at least one container associated to them.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B
chetan@Chaitanya-Raj:~/docker$ sudo apt purge docker
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Package 'docker' is not installed, so not removed
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
chetan@Chaitanya-Raj:~/docker$ sudo apt autoremove
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
chetan@Chaitanya-Raj:~/docker$ sudo apt autoclean
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
chetan@Chaitanya-Raj:~/docker$
```

To remove **docker-desktop** from **Windows OS** Subsystem for Linux (WSL), Run command prompt as administrator.

- Run command to see which app are running under WSL:

wsl -l -v

```
C:\Windows\System32>
C:\Windows\System32>wsl -l -v
  NAME              STATE           VERSION
* Ubuntu            Running         2
  Ubuntu-22.04      Running         2
  docker-desktop    Stopped         2

C:\Windows\System32>
```

- Type below command to uninstall Docker:

wslconfig /unregister docker-desktop

//OR//

wsl --unregister docker-desktop

```
C:\Windows\System32>wsl -l -v
  NAME              STATE           VERSION
* Ubuntu            Running         2
  Ubuntu-22.04      Running         2
  docker-desktop    Stopped         2

C:\Windows\System32>wslconfig /unregister docker-desktop
Unregistering.
The operation completed successfully.

C:\Windows\System32>wsl -l -v
  NAME            STATE           VERSION
* Ubuntu          Running         2
  Ubuntu-22.04    Running         2

C:\Windows\System32>
```

6 Basic Docker Commands

Now, let us look at some common Docker commands and their usage:

6.1 Docker PS

Command: `docker ps [options]`

Options:

-a, --all: Show all containers (default shows just running).

-q, --quiet: Only display container IDs.

--no-trunc: Don't truncate output.

-n, --last n: Show n last created containers (includes all states, default shows just running).

--latest, -l: Show the latest created container (includes all states, default shows just running).

--size, -s: Display total file sizes.

--format "TEMPLATE": Pretty-print containers using a Go template.

--filter FILTER: Filter output based on conditions provided (e.g., `docker ps --filter "status=running"`).

--format: Pretty-print containers using a Go template.

--format "TEMPLATE": Format the output using the given Go template.

Usage: Lists the currently running containers.

```
chetan@Chaitanya-Raj:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
chetan@Chaitanya-Raj:~$ |
```

6.2 Docker Pull

Command: `docker pull image_name`

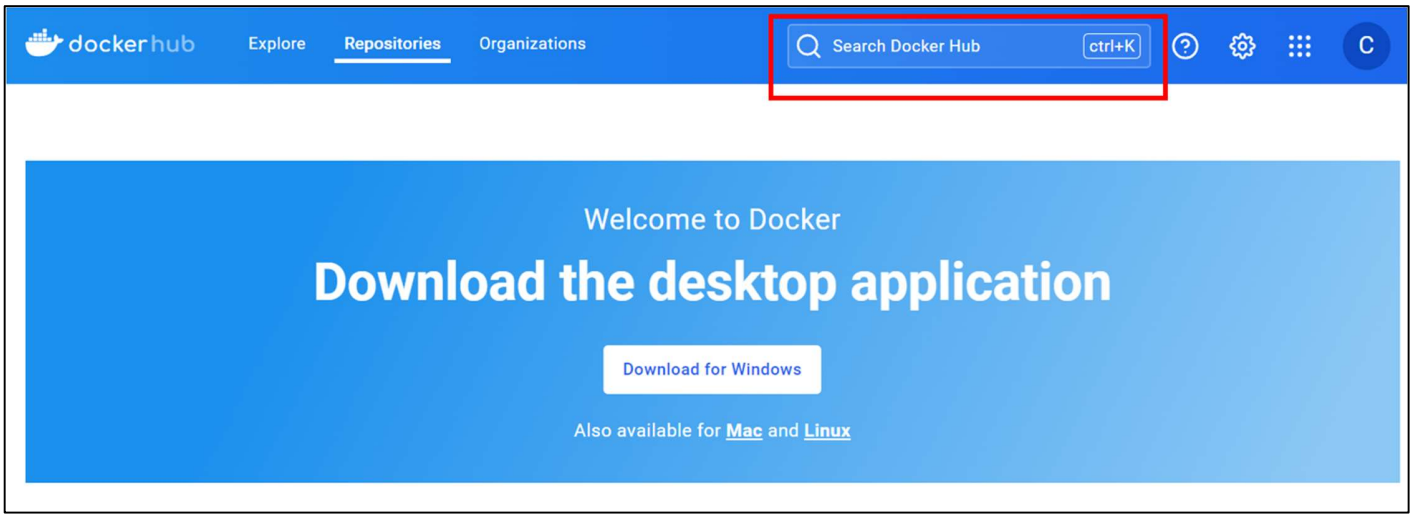
Usage: Downloads a Docker image from a registry.

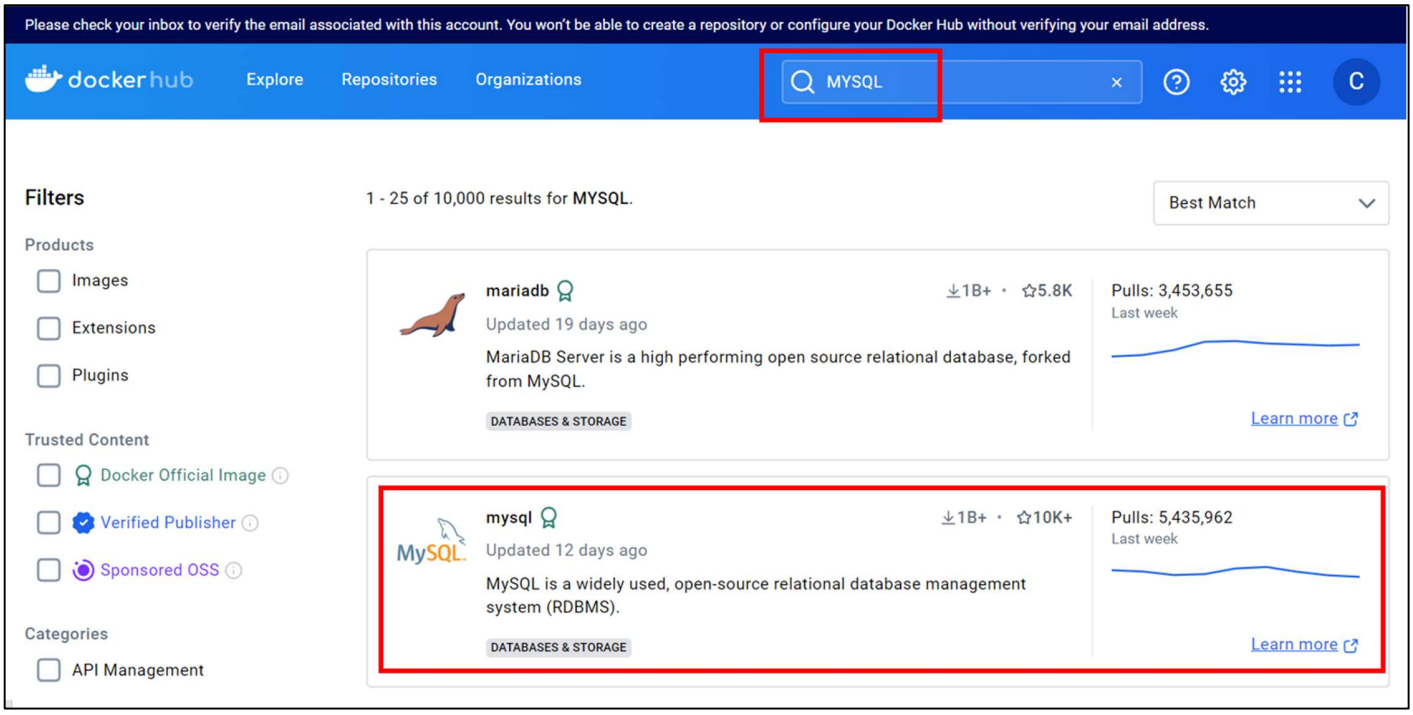
```
chetan@Chaitanya-Raj:~/docker$ docker pull mysql
Using default tag: latest
latest: Pulling from library/mysql
d9a40b27c30f: Pull complete
d948328c7651: Pull complete
c1e267313ede: Pull complete
7478f013875a: Pull complete
9221a2250289: Pull complete
d1f57baa52d5: Pull complete
35c3d30e8624: Pull complete
3d4d1dd0cca6: Pull complete
08bfe3f7d1c5: Pull complete
537e7daeeea3: Pull complete
Digest: sha256:72a37ddc9f839cfd84f1f6815fb31ba26f37f4c200b90e49607797480e3be446
Status: Downloaded newer image for mysql:latest
docker.io/library/mysql:latest
chetan@Chaitanya-Raj:~/docker$ |
```

Note!!!!

In command "**`docker pull`** `image_name`" image name can be taken from the DOCKER HUB GUI.

Log in to docker HUB GUI and SEARCH for the application package you want to pull from DOCKER HUB GUI.





Copy the pull command in the page given and paste in the linux terminal.

6.3 Docker Images

Command: `docker images [options]`

Usage: Lists all available Docker images on the local machine.

```
chetan@Chaitanya-Raj:~/docker$ docker images
REPOSITORY   TAG       IMAGE ID       CREATED       SIZE
mysql        latest    5cde95de907d   2 weeks ago   586MB
chetan@Chaitanya-Raj:~/docker$
chetan@Chaitanya-Raj:~/docker$
```

6.4 Docker Run

Command: `docker run [options] image_name:tag`

Usage: Creates and runs a container based on a specified image.

Note: Different Applications have different docker run options like -i, -d, -p, -t, etc.

1. **Interactive Mode (`-i, --interactive`):**

   `-i, --interactive`: Keep STDIN open even if not attached.

   `docker run -i -t ubuntu bash`

   Starts a new container interactively with a `bash` shell.

2. **TTY Mode (`-t, --tty`):**

   `-t, --tty`: Allocate a pseudo-TTY.

   `docker run -it ubuntu`

   Allocates a pseudo-TTY (terminal) and connects it to the container's STDIN, allowing you to interact with the container's shell.

3. **Detached Mode (`-d, --detach`):**

   `-d, --detach`: Run container in background and print container ID.

   `docker run -d nginx`

   Runs the container in the background (detached mode), printing the container ID.

4. **Name Container (`--name`):**

   `--name`: Assign a name to the container.

   `docker run --name my-container nginx`

   Assigns the name `my-container` to the container running the `nginx` image.

5. **Publish Ports (`-p, --publish`):**

`-p, --publish`: Publish a container's port(s) to the host.

```
docker run -p 8080:80 nginx
```

Publishes port 80 from the container to port 8080 on the host.

6. **Mount Volumes (`-v, --volume`)**:

   `-v, --volume`: Bind mount a volume.

```
docker run -v /host/path:/container/path nginx
```

Mounts a host directory (`/host/path`) into the container at `/container/path`.

7. **Environment Variables (`-e, --env`)**:

   `-e, --env`: Set environment variables.

```
docker run -e MYSQL_ROOT_PASSWORD=password mysql
```

Sets the environment variable `MYSQL_ROOT_PASSWORD` to `password` for the `mysql` container.

FOR EXAMPLE:

```
chetan@Chaitanya-Raj:~/docker$ docker run -e MYSQL_ROOT_PASSWORD=Infy@1028517 mysql
2024-07-21 15:59:27+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 9.0.0-1.el9
started.
2024-07-21 15:59:27+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
2024-07-21 15:59:27+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 9.0.0-1.el9
started.
2024-07-21 15:59:27+00:00 [Note] [Entrypoint]: Initializing database files
2024-07-21T15:59:27.712820Z 0 [System] [MY-015017] [Server] MySQL Server Initialization - start.
2024-07-21T15:59:27.714128Z 0 [System] [MY-013169] [Server] /usr/sbin/mysqld (mysqld 9.0.0)
initializing of server in progress as process 81
2024-07-21T15:59:27.722706Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
2024-07-21T15:59:27.967225Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
2024-07-21T15:59:28.930371Z 6 [Warning] [MY-010453] [Server] root@localhost is created with an
empty password ! Please consider switching off the --initialize-insecure option.
2024-07-21T15:59:30.675576Z 0 [System] [MY-015018] [Server] MySQL Server Initialization - end.
2024-07-21 15:59:30+00:00 [Note] [Entrypoint]: Database files initialized
2024-07-21 15:59:30+00:00 [Note] [Entrypoint]: Starting temporary server
2024-07-21T15:59:30.797074Z 0 [System] [MY-015015] [Server] MySQL Server - start.
2024-07-21T15:59:31.024473Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 9.0.0)
starting as process 133
2024-07-21T15:59:31.035365Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
2024-07-21T15:59:31.192902Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
2024-07-21T15:59:31.418660Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
2024-07-21T15:59:31.418725Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to
support TLS. Encrypted connections are now supported for this channel.
2024-07-21T15:59:31.423495Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file:
Location '/var/run/mysqld' in the path is accessible to all OS users. Consider choosing a different
directory.
2024-07-21T15:59:31.459220Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Socket:
/var/run/mysqld/mysqlx.sock
2024-07-21T15:59:31.459284Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for
connections. Version: '9.0.0'  socket: '/var/run/mysqld/mysqld.sock'  port: 0  MySQL Community
Server - GPL.
2024-07-21 15:59:31+00:00 [Note] [Entrypoint]: Temporary server started.
'/var/lib/mysql/mysql.sock' -> '/var/run/mysqld/mysqld.sock'
Warning: Unable to load '/usr/share/zoneinfo/iso3166.tab' as time zone. Skipping it.
Warning: Unable to load '/usr/share/zoneinfo/leap-seconds.list' as time zone. Skipping it.
Warning: Unable to load '/usr/share/zoneinfo/leapseconds' as time zone. Skipping it.
Warning: Unable to load '/usr/share/zoneinfo/tzdata.zi' as time zone. Skipping it.
Warning: Unable to load '/usr/share/zoneinfo/zone.tab' as time zone. Skipping it.
Warning: Unable to load '/usr/share/zoneinfo/zone1970.tab' as time zone. Skipping it.

2024-07-21 15:59:32+00:00 [Note] [Entrypoint]: Stopping temporary server
2024-07-21T15:59:32.835818Z 10 [System] [MY-013172] [Server] Received SHUTDOWN from user root.
Shutting down mysqld (Version: 9.0.0).
```

```
2024-07-21T15:59:33.826132Z 0 [System] [MY-010910] [Server] /usr/sbin/mysqld: Shutdown complete
(mysqld 9.0.0)  MySQL Community Server - GPL.
2024-07-21T15:59:33.826212Z 0 [System] [MY-015016] [Server] MySQL Server - end.
2024-07-21 15:59:33+00:00 [Note] [Entrypoint]: Temporary server stopped

2024-07-21 15:59:33+00:00 [Note] [Entrypoint]: MySQL init process done. Ready for start up.

2024-07-21T15:59:33.850319Z 0 [System] [MY-015015] [Server] MySQL Server - start.
2024-07-21T15:59:34.027025Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 9.0.0)
starting as process 1
2024-07-21T15:59:34.032494Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
2024-07-21T15:59:34.171669Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
2024-07-21T15:59:34.303989Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
2024-07-21T15:59:34.304030Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to
support TLS. Encrypted connections are now supported for this channel.
2024-07-21T15:59:34.306425Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file:
Location '/var/run/mysqld' in the path is accessible to all OS users. Consider choosing a different
directory.
2024-07-21T15:59:34.323433Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Bind-
address: '::' port: 33060, socket: /var/run/mysqld/mysqlx.sock
2024-07-21T15:59:34.323459Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for
connections. Version: '9.0.0'  socket: '/var/run/mysqld/mysqld.sock'  port: 3306  MySQL Community
Server - GPL.

2024-07-21T16:05:16.505164Z 0 [System] [MY-013172] [Server] Received SHUTDOWN from user <via user
signal>. Shutting down mysqld (Version: 9.0.0).
2024-07-21T16:05:17.764934Z 0 [System] [MY-010910] [Server] /usr/sbin/mysqld: Shutdown complete
(mysqld 9.0.0)  MySQL Community Server - GPL.
2024-07-21T16:05:17.764966Z 0 [System] [MY-015016] [Server] MySQL Server - end.
chetan@Chaitanya-Raj:~/docker$
chetan@Chaitanya-Raj:~/docker$
```

8. **Container Resource Limits (`--memory`, `--cpus`)**:

   `--memory`: Limit memory usage.

   `--cpus`: Limit CPU usage.

   ```
   docker run --memory 512m --cpus 0.5 nginx
   ```

   Limits the memory to 512 MB and CPU usage to 0.5 CPUs for the `nginx` container.

   Example:

   ```
   docker run -d --name my-nginx -p 8080:80 nginx
   ```

6.5 Docker PS

Command: **`docker ps`** `[options]`
Usage: Lists the currently running containers.

```
chetan@Chaitanya-Raj:~/docker$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS         PORTS                   NAMES
103f846de8c8   mysql     "docker-entrypoint.s…"   2 minutes ago  Up 2 minutes   3306/tcp, 33060/tcp     compassionate_elbakyan
chetan@Chaitanya-Raj:~/docker$
```

Options:
-a, --all: Show all containers (default shows just running).
-q, --quiet: Only display container IDs.
--no-trunc: Don't truncate output.
-n, --last n: (Here n = number): Show “n” last created containers (includes all states, default shows just running).

--latest, -l: Show the latest created container (includes all states, default shows just running).
--size, -s: Display total file sizes.
--format "TEMPLATE": Pretty-print containers using a Go template.
--filter FILTER: Filter output based on conditions provided (e.g., `docker ps --filter "status=running"`).
--format: Pretty-print containers using a Go template.
--format "TEMPLATE": Format the output using the given Go template.

6.6 Docker Stop

Command: **`docker stop`** `container_id`

Usage: Stops a running container.

```
chetan@Chaitanya-Raj:~/docker$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS                   NAMES
103f846de8c8   mysql     "docker-entrypoint.s…"   2 minutes ago   Up 2 minutes   3306/tcp, 33060/tcp     compassionate_elbakyan
chetan@Chaitanya-Raj:~/docker$ docker stop 103f846de8c8
103f846de8c8
chetan@Chaitanya-Raj:~/docker$
```

6.7 Docker RM (Remove Container)

Command: **`docker rm`** `container_id` ***//OR//*** **`docker remove`** `container_id`

Usage: Removes a stopped container.

```
chetan@Chaitanya-Raj:~/docker$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS                     PORTS     NAMES
103f846de8c8   mysql     "docker-entrypoint.s…"   9 minutes ago   Exited (0) 3 minutes ago             compassionate_elbakyan
chetan@Chaitanya-Raj:~/docker$ docker rm 103f846de8c8
103f846de8c8
chetan@Chaitanya-Raj:~/docker$ docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
chetan@Chaitanya-Raj:~/docker$
```

6.8 Docker RMI (Remove Image)

Command: **`docker rmi`** `image_name:tag`

Usage: Removes a Docker image from the local machine.

```
chetan@Chaitanya-Raj:~/docker$ docker images
REPOSITORY   TAG       IMAGE ID       CREATED       SIZE
mysql        latest    5cde95de907d   2 weeks ago   586MB
chetan@Chaitanya-Raj:~/docker$ docker rmi mysql:latest
Untagged: mysql:latest
Untagged: mysql@sha256:72a37ddc9f839cfd84f1f6815fb31ba26f37f4c200b90e49607797480e3be446
Deleted: sha256:5cde95de907d193090219b5b825b60439321cca331fb63bad4e2441a4c9b5f7a
Deleted: sha256:87755ece0fc4afeae41690d51cdc7603a11fdf8c337386670891bc70a96d318e
Deleted: sha256:bb55ff86eb2f2323b1082cbecc287a00c5b555305a78beb6e1fadedc2edf6ee9
Deleted: sha256:6cfeea99c3d8f89374ee4fe1a6d92c2ef1dbe5e408f424967dbae13636ee4ea0
Deleted: sha256:53b04931a5b66d07980b83065347332b2a3451f01ccb8e2164c527e904133b1b
Deleted: sha256:9e5c79587cdad26d8f83eb1c17fa95c52fa7be00e69196672a796524eac43df3
Deleted: sha256:6828e70104255416c757dc4278d232b2798055dea72baae824d53df9c964e958
Deleted: sha256:988ef96e23f97606f691b80db04446b3b21268ccb0eef1010cef23ebfb82eb9d
Deleted: sha256:782a7515fa1a1c8bf955fb72fe5d3112e9dd447efb020b91a68860c0537b2bf0
Deleted: sha256:6247b710a5e69835fba492b845eb5157f69a8496792b8aed2c15d68e866ae65a
Deleted: sha256:2606c15a4838dfb909dab29f47c601a797f657f51a35005bd06d01da891d813c
chetan@Chaitanya-Raj:~/docker$
chetan@Chaitanya-Raj:~/docker$
chetan@Chaitanya-Raj:~/docker$ docker images
REPOSITORY   TAG       IMAGE ID   CREATED   SIZE
chetan@Chaitanya-Raj:~/docker$
```

6.9 Docker Build

Command: `docker build -t image_name:tag .`

Usage: Builds a Docker image from a Dockerfile in the current directory.

6.10 Docker Push

Command: `docker push image_name:tag`

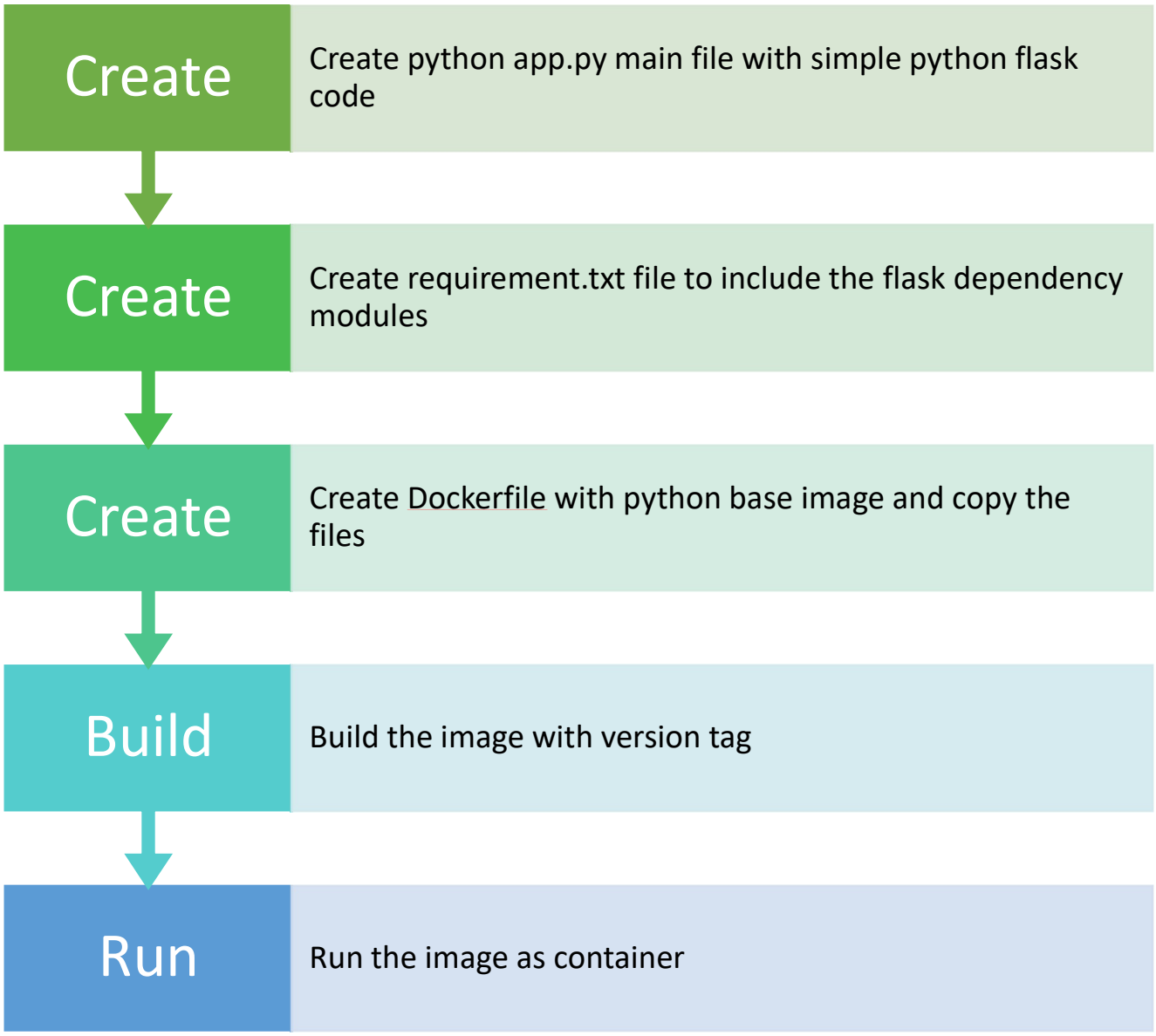
Usage: Uploads a Docker image to a registry.

6.11 Docker Exec

Command: `docker exec [options] container_id command`

Usage: Executes a command inside a running container.

7 Building a Python Web App



8 ERROR AND ITS SOLUTION

8.1 Unauthorized: incorrect username or password

LOGIN TO DOCKER USING DOCKER LOGIN COMMAND.

It seems like Docker is unable to pull the MySQL image because it's encountering an authorization issue with the Docker registry. The error message "unauthorized: incorrect username or password" suggests that Docker is not able to authenticate your credentials correctly.

Log in to Docker:

`docker login`

```
chetan@Chaitanya-Raj:~/docker$ which docker
/usr/bin/docker
chetan@Chaitanya-Raj:~/docker$ docker pull mysql
Using default tag: latest
Error response from daemon: Head "https://registry-1.docker.io/v2/library/mysql/manifests/latest": unauthorized: incorre
ct username or password
chetan@Chaitanya-Raj:~/docker$ docker login
Authenticating with existing credentials...
Stored credentials invalid or expired
Log in with your Docker ID or email address to push and pull images from Docker Hub. If you don't have a Docker ID, head
 over to https://hub.docker.com/ to create one.
You can log in with your password or a Personal Access Token (PAT). Using a limited-scope PAT grants better security and
 is required for organizations using SSO. Learn more at https://docs.docker.com/go/access-tokens/

Username (chaitanyarajndn): chaitanyarajndn
Password:
Login Succeeded
chetan@Chaitanya-Raj:~/docker$ docker pull mysql
Using default tag: latest
latest: Pulling from library/mysql
d9a40b27c30f: Pull complete
d948328c7651: Pull complete
c1e267313ede: Pull complete
7478f013875a: Pull complete
9221a2250289: Pull complete
d1f57baa52d5: Pull complete
35c3d30e8624: Pull complete
3d4d1dd0cca6: Pull complete
08bfe3f7d1c5: Pull complete
537e7daeeea3: Pull complete
Digest: sha256:72a37ddc9f839cfd84f1f6815fb31ba26f37f4c200b90e49607797480e3be446
Status: Downloaded newer image for mysql:latest
docker.io/library/mysql:latest
chetan@Chaitanya-Raj:~/docker$
```

9